The Journal of Open Source Software

CAWSR: Carla-AutoWare Scenario Runner

David Gasinski [1], **Olek Osikowicz** [1], **Gwilym Rutherford** [1], and **Donghwan Shin** [1]

1 The University of Sheffield

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

CAWSR (**C**ARLA-**A**uto**W**are-**S**cenario **R**unner) facilitates the simulation-based testing of the open-source autonomous driving system, Autoware, within CARLA, the state-of-the-art open-source driving simulator. Building on existing tools, this project introduces a research-oriented testing framework for the execution of complex driving scenarios, as well as supporting implementation of a wide range of verification strategies.

Statement of Need

Verifying Autonomous Driving Systems (ADS) is a critical step before they can be deployed. However, relying only on real-world testing is too expensive, inefficient, and potentially dangerous. Consequently, simulation-based testing has become essential, allowing researchers to safely test driving agents against critical situations at scale. Among these tools, CARLA (Dosovitskiy et al., 2017) has become the de-facto standard in the research community due to its rich ecosystem of open-source tools, benchmarks, and documentation.

Currently, the standard for evaluating ADS in CARLA is the CARLA Leaderboard and its engine, Scenario Runner (SR) (CARLA, 2025). This framework is typically used to test "black-box" driving agents, such as those based on Vision Language Models or Reinforcement Learning (DS: add refs here). By running a set of predefined, challenging driving scenarios, researchers can systematically assess agent performance using common metrics like driving score, infractions, and route completion. However, applying this testing framework to industry-grade ADS, such as Autoware (Kato et al., 2018) or Apollo (Baidu, 2017), remains difficult. Although communication bridges exist between CARLA and these systems (Guardstrikelab, 2023; Kaljavesi et al., 2024), they lack native support for scenario execution engines, which limits their utility for scenario-based testing.

This gap has created a significant bottleneck for the research community. Previously, researchers developing scenario generation algorithms mainly relied on combining Apollo with the LGSVL simulator (Rong et al., 2020). However, LGSVL is now outdated, with official support ending in January 2022. This leaves many researchers without a suitable industry-grade "subject" for evaluating their algorithms. While recent tools like PCLA (Tehrani et al., 2025) attempt to simplify deploying Autoware (and other ADS implementations) into CARLA, they focus primarily on simplifying the ADS implementations and abstracting the setup process across different CARLA versions. They lack the deep integration required between the agent and simulator to execute complex, route-based scenarios.

CAWSR aims to bridge this gap by enabling the evaluation of Autoware in complex driving scenarios within CARLA. By building on the established CARLA platform, this work provides a modern replacement for the outdated Apollo/LGSVL workflow. It also allows Autoware to be directly compared with state-of-the-art research agents on the CARLA Leaderboard.

DRAFT

Effective ADS verification requires the ability to systematically explore the operational design domain. To support this, CAWSR provides a flexible interface for algorithmic scenario generation. This facilitates a wide range of verification strategies based on common metrics, such as the CARLA Leaderboard's driving score (CARLA Team, 2024).

Lastly, it is worth noting that simulators can often introduce unintended nondeterminism, which leads to inconsistent test results (Chance et al., 2022; Osikowicz et al., 2025). Therefore, CAWSR is designed to minimise such nondeterminism throughout the evaluation pipeline.

Tool Overview

CAWSR is a fully synchronous testing framework that directly integrates the CARLA simulator, Scenario Runner (as the scenario executor), and Autoware (as the System Under Test) to facilitate autonomous driving testing research. The tool is distributed as a Docker container designed and currently supports two modes of operation:

1. *Scenario Generation Mode:* Enables the dynamic generation and execution of scenarios (e.g. iterative scenario generation) provided by a user-defined algorithm. This is particularly useful for assessing the performance of new simulation-based ADS testing techniques.
2. *Benchmark Mode:* Allows the execution of a predefined set of scenario definitions provided by the user. This is useful for standardised evaluations and comparisons between different driving agents.

The evaluation pipeline is engineered to be fully synchronous, minimising unintentional nondeterminism to facilitate reproducible results. However, it is noted that minor variations may still persist due to inherent non-determinism in upstream dependencies, such as the driving simulator or the driving agent itself (Chance et al., 2022; Osikowicz et al., 2025).

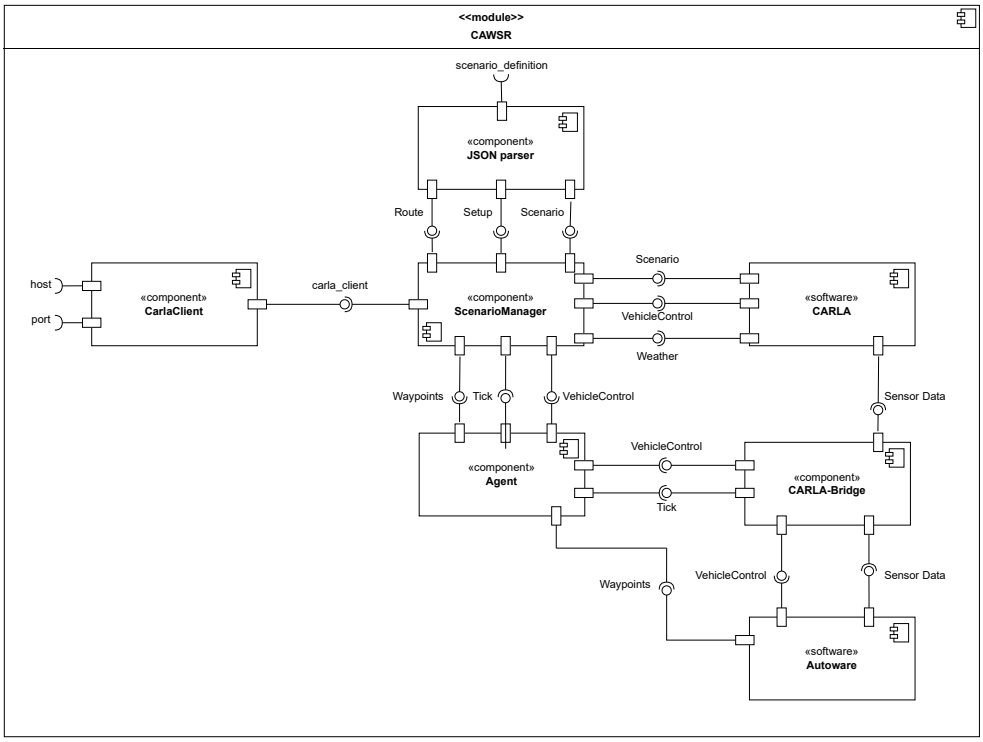

Figure 1: Internal component diagram of CAWSR.

Figure 1 shows the overall architecture of CAWSR.

(**David**: This is the first format, although i feels a bit short and doesn't elaborate on each component much and how they interact together.)

Table 1: Function of the key modules in CAWSR.

Component	Function
CarlaClient	A native class of the CARLA PythonAPI, facilitating interaction with the simulator.
JSON parser	A module which converts the *scenario_defintion* into XML behaviour trees, used for scenrario execution by the ScenarioManager.
ScenarioManager	Scenario Runner module which handles the execution of scenarios through behaviour trees.
CarlaBridge	This module handles the transformation of sensor data from CARLA to ROS. At each step, data is sent to Autoware, responding with a control command that is applied to the ego vehicle in simulation.
Agent	Monitors the internal state of key Autoware modules, handling the communication between CAWSR and Autoware. Responsible for sending over route information during initialisation and executing the CarlaBridge at each step.

(**David**: Second format, more text and less readability than the first, but offers significanlty more detail.)

Each component of CAWSR plays a fundamental role in the overall architecture of the framework. The **CarlaClient** is a native class of the CARLA PythonAPI. Accepting a *host* IP and *port*, it creates a TCP connection to the simulator and allows interaction with its interval server, enabling CAWSR modules to extract information and spawn entities. This forms the basis of CAWSR and the single point of communication between the framework and CARLA.

The **JSON parser** converts the JSON *scenario_definition* into a behaviour tree (BT), extracting information about the *route* and events that trigger along it. The fundamental elements of the behaviour trees are assembled from *Atomic Behaviours* and *Atomic Conditions*, introduced by Scenario Runner. These represent individual behaviours and variables within CARLA, such as spawning a pedestrian, serving as the building blocks of scenarios.

ScenarioManager handles the initial setup and execution loop, using the CarlaClient to spawn the relevant scenario entities. At each step, the scenario behaviour tree is executed, calculating updated states for each actor and triggering relevant conditions. Through the CarlaClient, a single *tick* (step) is sent to CARLA, incrementing its internal clock and generating a new snapshot of the simulation. This snapshot is sent to the **Agent** module, monitoring the internal state of Autoware modules and sending information about the route. During initialisation, the agent instantiates a connection with Autoware through ROS. Once this connection is established, at each execution step, the **CarlaBridge** extracts data through the snapshot. Sensor data is transformed into Autoware's coordinate system and published to the relevant modules. Once processed, Autoware updates its internal state and responds with a control command, applying it to the ego vehicle in simulation.

The internal loop within ScenarioManager continues executing until one of the following termination conditions is met, as defined by the CARLA Leaderboard evaluation criteria (CARLA Team, 2024), as shown in Table 2.

DRAFT

Table 2: Termination Criteria of each scenario within CAWSR.

Criteria	Description
Route_Completion	Agent reached the end of the route.
Actor_Blocked	Agent is blocked, not moving for 180s.
Simulation_Timeout	No client-server communication can be established for 30s.

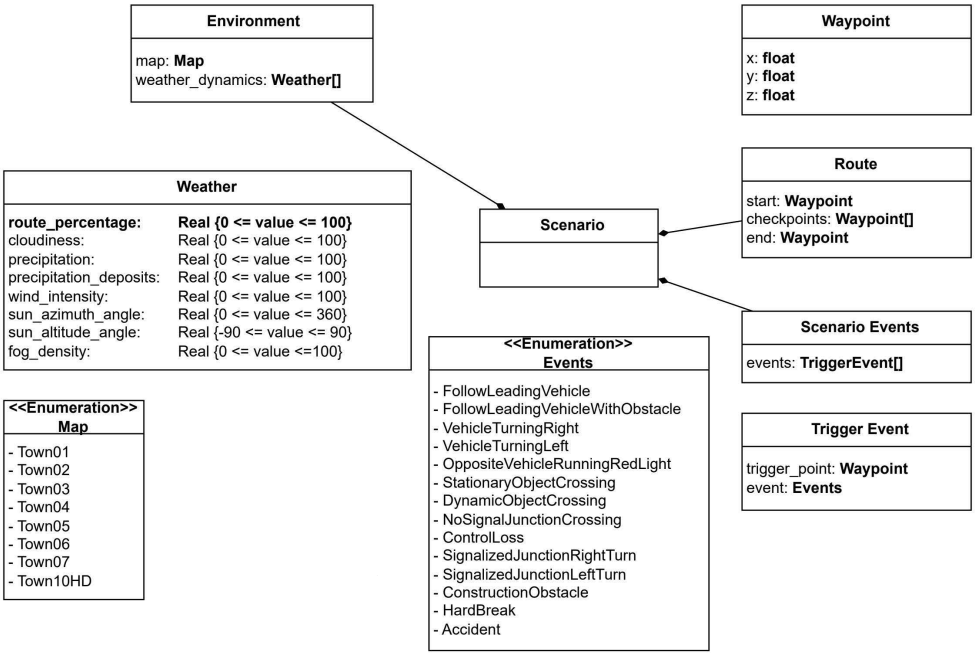

Figure 2: Scenario definition domain model.

To facilitate development, we introduce a new domain model for the definition of route-based scenarios within CARLA, described in Figure 3, alongside a `JSON` implementation. This model is based on the format introduced by Scenario Runner, facilitating support between both frameworks.

Conclusion

To summarise, `CAWSR` provides ADS testing research community an easy to use Autoware evaluation pipeline. We hope that this work can facilitate the evaluation of new testing approaches on a state of the art driving system.

Acknowledgements

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-02218761, 50%) and by the Engineering and Physical Sciences Research Council (EPSRC) [EP/Y014219/1].

References

Baidu. (2017). *Apollo: Open Source Autonomous Driving*. https://github.com/ApolloAuto/apollo.

CARLA. (2025). *Scenario Runner: Traffic Scenario Definition and Execution Engine for CARLA*. GitHub; https://github.com/carla-simulator/scenario_runner. https://github.com/carla-simulator/scenario_runner

CARLA Team. (2024). *Get started with leaderboard 2.0. CARLA autonomous driving leaderboard*. https://leaderboard.carla.org/get_started_v2_0/

Chance, G., Ghobrial, A., McAreavey, K., Lemaignan, S., Pipe, T., & Eder, K. (2022). On determinism of game engines used for simulation-based autonomous vehicle verification. *IEEE Transactions on Intelligent Transportation Systems*, *23*(11), 20538–20552. https://doi.org/10.1109/TITS.2022.3177887

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An open urban driving simulator. *Proceedings of the 1st Annual Conference on Robot Learning*, 1–16.

Guardstrikelab. (2023). *carla_apollo_bridge: Data and Control Bridge for Apollo and Carla*. GitHub; https://github.com/guardstrikelab/carla_apollo_bridge.

Kaljavesi, G., Kerbl, T., Betz, T., Mitkovskii, K., & Diermeyer, F. (2024). *CARLA-autoware-bridge: Facilitating autonomous driving research with a unified framework for simulation and module development*. 224–229. https://doi.org/10.1109/iv55156.2024.10588623

Kato, S., Tokunaga, S., Maruyama, Y., Maeda, S., Hirabayashi, M., Kitsukawa, Y., Monrroy, A., Ando, T., Fujii, Y., & Azumi, T. (2018). Autoware on board: Enabling autonomous vehicles with embedded systems. *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*, 287–296.

Osikowicz, O., McMinn, P., & Shin, D. (2025). Empirically evaluating flaky tests for autonomous driving systems in simulated environments. *2025 IEEE/ACM International Flaky Tests Workshop (FTW)*, 13–20.

Rong, G., Shin, B. H., Tabatabaee, H., Lu, Q., Lemke, S., Možeiko, M., Boise, E., Uhm, G., Gerow, M., Mehta, S., Agafonov, E., Kim, T. H., Sterner, E., Ushiroda, K., Reyes, M., Zelenkovsky, D., & Kim, S. (2020). LGSVL simulator: A high fidelity simulator for autonomous driving. *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, 1–6. https://doi.org/10.1109/ITSC45102.2020.9294422

Tehrani, M. J., Kim, J., & Tonella, P. (2025). PCLA: A framework for testing autonomous agents in the CARLA simulator. *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 1040–1044.

DRAFT